

SafeDig AI — Codebase File-by-File Technical Guide

Complete Architectural Walkthrough of all 85+ Source Modules & Subsystems

Classification: Engineering Reference & Trainee Onboarding Handbook

Compliance Standard: UK Health and Safety Guidance 47 (HSG47) & CDM 2015

1. Directory Tree & Architecture Map

SafeDig_AG/	
alembic/	# Database schema versioning & migration scripts
Data/	# Statutory utility enquiry test jobs & sample packs
Documentation/	# Engineering specifications & compliance PDFs
qa_output/	# Output forensic packages, crop PNGs & job reports
src/	# Core Python 3.11 application codebase
agent/	# Advisory LLM models & service integration
aoi/	# Area of Interest boundary detection & validation
api/	# FastAPI web server, routes & static HTML5 console
batch/	# Batch job queues, scanner & background workers
config/	# Pydantic BaseSettings singleton & structured logging
cv/	# OpenCV rasterization, HSV color masks & contours
db/	# SQLite engine, SQLAlchemy models & persistence
detection/	# Dual-engine spatial detection coordinator
documents/	# Document classification & resolution services
domain/	# Strict Pydantic domain models & system enums
eval/	# Ground-truth dataset evaluation & accuracy metrics
evidence/	# Forensic crop generator, overlays & completeness
index/	# Excel index.xlsx ingestion, reconciler & validator
ingestion/	# Fuzzy inventory matcher, hasher & manifest parser
legends/	# Cartographic legend registry, detector & resolver
ocr/	# Tesseract OCR hazard label scanner
orchestration/	# LangGraph state machine DAG, nodes & state dict
pdf/	# PyMuPDF extractor, inspector & page renderer
policy/	# The 17 Mandatory Release Gates & PolicyEngine
providers/	# Statutory undertaker registry (UKPN, SGN, etc.)
qa/	# QA workspace builder & human review disposition
reconciliation/	# Shapely 2D spatial buffer & intersection engine
reporting/	# Compliance report generator & summary exports
spatial/	# Coordinate transformations & spatial math
utils/	# Self-healing path resolver, security & telemetry
vector/	# Native PDF vector path analyzer & geometry math
warnings/	# Statutory hazard catalogue & HSG47 threshold tables
pipeline.py	# End-to-end pipeline CLI & batch runner
tests/	# PyTest unit, integration & regression test suites
tools/	# Documentation generators & maintenance utilities
safedig.db	# Forensic SQLite audit database (WAL mode)

2. Root Configuration & Deployment Files

- pyproject.toml**: Defines project build-system ([setuptools](#)), metadata, Python 3.11 environment constraints, pytest configuration (`asyncio_mode = "auto"`), and test discovery markers.
- requirements.txt**: Production runtime dependencies: [fastapi](#), [uvicorn](#), [pymupdf](#), [shapely](#), [opencv-python-headless](#), [pandas](#), [openpyxl](#), [pydantic](#), [pydantic-settings](#), [sqlalchemy](#), [langgraph](#), [langchain-core](#), [httpx](#), [reportlab](#).
- requirements-dev.txt**: Developer testing tools: [pytest](#), [pytest-asyncio](#), [pytest-cov](#), [black](#), [flake8](#), [mypy](#).
- .env** / **.env.example**: Application environment file configuring `SAFE_MODE=True`, `DATABASE_URL=sqlite:///safedig.db`, `OLLAMA_BASE_URL=http://localhost:11434`, `OLLAMA_MODEL=qwen2.5:7b`, server bind host/port (`127.0.0.1:8000`), and log verbosity.
- Dockerfile**: Production container build based on [python:3.11-slim](#). Installs system C-libraries ([libgl-mesa-glx](#), [libglvnd](#)), sets up non-root execution user, and runs Uvicorn ASGI workers.
- docker-compose.yml**: Docker service definition orchestrating SafeDig web container and local Ollama inference service.
- nginx-safedig.conf**: Nginx reverse proxy configuration providing SSL termination, client max body size (100MB for CAD uploads), and static caching.

- ****safedig.service****: Linux systemd unit file for daemonized production background execution with automatic restarts.
- ****alembic.ini****: Configuration file for Alembic database migration environment.
- ****run_production.bat / run_production.sh****: Launch scripts for Windows and Linux production environments.
- ****run_e2e_test.py****: Standalone end-to-end regression test script exercising full ingestion, spatial detection, policy evaluation, and database write across sample packs.

3. Pipeline Entry Point ([src/pipeline.py](#))

- **File**: [src/pipeline.py](#)
- **Role**: Primary standalone execution script for CLI operations and scheduled background batch jobs.
- **Key Function**: `run_pipeline(job_dir: Path, safe_mode: bool = True) -> PipelineResult`
- **Workflow**:
 1. Invokes [SafePathResolver](#) to validate job directory.
 2. Parses [index.xlsx](#) via [IndexParser](#).
 3. Builds LangGraph state machine from [src/orchestration/graph.py](#).
 4. Runs graph to completion, handling exceptions per document.
 5. Commits audit records to SQLite [safedig.db](#).
 6. Prints high-contrast terminal summary table and returns exit code (0 = success, 1 = unhandled failure).

4. Configuration & Logging ([src/config/](#))

- ****src/config/settings.py****:
 - Implements [Settings\(BaseSettings\)](#) singleton.
 - Manages environment variables: [safe_mode](#), [debug](#), [database_url](#), [ollama_base_url](#), [ollama_model](#), [evidence_dir](#), [log_level](#).
 - Enforces type safety and validates directory creation on startup.
- ****src/config/logging.py****:
 - Enterprise logging configuration.
 - Configures rotating file handlers ([logs/safedig.log](#), max 20MB, 5 backups) and colored console output with ISO 8601 timestamps.

5. Domain Models ([src/domain/](#))

The domain layer enforces strict data contracts using Pydantic v2 models:

- ****src/domain/enums.py****:
 - **Decision**: [AUTO_CLEAR](#), [HUMAN_REVIEW](#), [BLOCKED](#).
 - **PDFModality**: [VECTOR](#), [RASTER](#), [HYBRID](#), [UNREADABLE](#).
 - **DocumentResolutionStatus**: [UNIQUE](#), [EXCLUDED](#), [AMBIGUOUS](#), [NOT_FOUND](#).
 - **ReconciliationOutcome**: [CONFIRMED_CLEAN](#), [MATCH](#), [MISSED_WARNING](#), [POSSIBLE_FALSE_POSITIVE](#).
 - **Severity**: [CRITICAL](#), [HIGH](#), [MEDIUM](#), [LOW](#), [INFO](#).
- ****src/domain/index_record.py****: Encapsulates a parsed line from [index.xlsx](#): utility name, declared status, boundary description, and raw provider comments.
- ****src/domain/document.py****: Represents a physical PDF drawing: file path, SHA-256 hash, page count, mediabox dimensions, and modality.
- ****src/domain/aoi.py****: Models the excavation Area of Interest polygon, Shapely geometry, coordinate vertex array, and validation flags.
- ****src/domain/asset.py****: Represents detected underground utilities: asset class (electric, gas, water, telecom), coordinate line string, voltage/pressure attributes.
- ****src/domain/detection.py****: Output data model from spatial detection engines containing extracted features and bounding boxes.
- ****src/domain/legend.py****: Cartographic legend profile model: RGB color bands, Delta-E tolerances, stroke widths, and dash patterns.
- ****src/domain/warning.py****: Domain model for statutory warnings, voltage classes, and HSG47 minimum standoff distances.
- ****src/domain/reconciliation.py****: Holds 2D geometric intersection calculations: outcome enum, intersecting length, proximity distance, and hazard severity.
- ****src/domain/policy.py****: Models individual [GateCheck](#) evaluations and composite [PolicyResult](#) decisions.

- **`**src/domain/evidence.py**`**: Defines **EvidencePackage**: paths to cropped AOI images, whole-sheet views, neon overlays, and completeness status.
- **`**src/domain/audit.py**`**: Immutable audit record model containing SHA-256 signatures, timestamps, gate states, and engineer review logs.

6. Ingestion & Index Subsystem ([src/index/](#) & [src/ingestion/](#))

- **`**src/index/parser.py**`**: Reads **index.xlsx**, extracts enquiry columns, cleans whitespace, normalizes utility names, and returns **List[IndexRecord]**.
- **`**src/index/validator.py**`**: Validates statutory index integrity: checks for required columns, duplicate utility entries, and empty fields.
- **`**src/index/reconciler.py**`**: Cross-references index entries against physical folder contents, verifying expected file counts.
- **`**src/ingestion/inventory.py**`**: Intelligent inventory matcher. Uses regex tokenization and fuzzy similarity scoring to link index records to authoritative CAD drawings, filtering out non-map leaflets.
- **`**src/ingestion/classifier.py**`**: Distinguishes between engineering plans, summary letters, hospital diagrams, and safety pamphlets.
- **`**src/ingestion/hasher.py**`**: Computes streaming SHA-256 cryptographic hashes of PDF files.
- **`**src/ingestion/manifest.py**`**: Generates and parses JSON ingestion manifests for reproducible batch tracking.
- **`**src/documents/resolver.py**`**: High-level document resolution coordinator orchestrating parser, hasher, and inventory matcher.

7. PDF Processing Engine ([src/pdf/](#))

- **`**src/pdf/extractor.py**`**: Opens PDF documents via PyMuPDF. Extracts metadata, page text, and drawing vector streams.
- **`**src/pdf/inspector.py**`**: Analyzes drawing primitives (**re**, **qu**, **c**, **l**) and embedded XObject images to classify document modality (**VECTOR**, **RASTER**, **HYBRID**).
- **`**src/pdf/renderer.py**`**: Renders PDF pages into high-resolution 300 DPI OpenCV BGR numpy arrays for computer vision processing.

8. Area of Interest (AOI) Detection ([src/aoi/](#) & [src/spatial/](#))

- **`**src/aoi/detector.py**`**: Scans drawings for excavation boundaries (red/magenta/yellow lines). Detects CAD viewports (Thames Water 4-line boxes, BT raster frames) and extracts polygon vertices.
- **`**src/aoi/service.py**`**: Service wrapper validating AOI geometry, checking closure (**coords[0] == coords[-1]**), vertex count, and area thresholds.
- **`**src/spatial/coordinates.py**`**: Normalizes coordinates between unrotated PDF mediabox coordinates and visually rotated screen coordinates (0°, 90°, 180°, 270°).
- **`**src/spatial/engine.py**`**: Constructs statutory HSG47 standoff buffer polygons (500mm hand-dig, 3m HV electric, 15m HP gas).

9. Cartographic Legends & Warnings ([src/legends/](#), [src/warnings/](#), [src/providers/](#))

- **`**src/legends/registry.py**`**: Master registry of utility drawing legends. Maps statutory providers (UKPN, SGN, Cadent, etc.) to target RGB color bands, Delta-E tolerances ($\Delta E \leq 28.0$), and stroke widths.
- **`**src/legends/detector.py**`**: Automatically discovers on-sheet legend boxes by detecting multi-column symbol grids.
- **`**src/legends/resolver.py**`**: Resolves the appropriate legend profile for a given enquiry record and drawing style.
- **`**src/warnings/catalogue.py**`**: Master taxonomy of utility hazard rules, voltage levels (LV, 11kV, 33kV, 132kV), gas pressures, and HSG47 legal actions.
- **`**src/providers/base.py**`**: Abstract base class defining provider interface contracts.
- **`**src/providers/registry.py**`**: Registry normalizing free-text utility names to canonical provider entities.

10. Vector Analysis & Computer Vision ([src/vector/](#), [src/cv/](#), [src/ocr/](#), [src/detection/](#))

- **`**src/vector/analyzer.py**`**: Channel A Vector Engine. Traverses PyMuPDF drawing paths, filters out the AOI boundary, and matches line strokes against legend profiles.

- **`**src/vector/geometry.py**`**: Geometric helper utilities converting raw PDF path segments into Shapely **LineString** objects.
- **`**src/cv/color.py**`**: Channel B Vision Engine. Converts BGR images to HSV color space and applies threshold masks for utility color bands.
- **`**src/cv/morphology.py**`**: Morphological operations (dilation, closing) bridging dashed line gaps in scanned blueprints.
- **`**src/cv/template.py**`**: Template matching for standard utility symbol markers (valves, manholes, link boxes).
- **`**src/ocr/service.py**`**: Channel C OCR Booster. Scans AOI area for hazard text tokens (**11kV**, **33kV**, **HP GAS**).
- **`**src/detection/discovery.py**`**: Discovers candidate hazard features across vector and raster channels.
- **`**src/detection/engine.py**`**: Dual-engine coordinator fusing Channel A vector results and Channel B computer vision findings.

11. Spatial Reconciliation & Safety Policy (**`src/reconciliation/`** & **`src/policy/`**)

- **`**src/reconciliation/engine.py**`**: Evaluates 2D geometric intersections between the buffered AOI and detected utility features using Shapely GEOS topology. Classifies outcome (**CONFIRMED_CLEAN**, **MATCH**, **MISSED_WARNING**, **POSSIBLE_FALSE_POSITIVE**).
- **`**src/policy/gates.py`**: **Evaluates** The 17 Mandatory Release Gates** deterministically ensuring zero escaped hazards.
- **`**src/policy/engine.py**`**: Executes the hierarchical decision ladder routing documents to **AUTO_CLEAR**, **HUMAN_REVIEW**, or **BLOCKED**.

12. LangGraph Orchestration & Advisory AI (**`src/orchestration/`** & **`src/agent/`**)

- **`**src/orchestration/state.py**`**: Defines typed state dictionary (**PipelineState**) flowing between pipeline nodes.
- **`**src/orchestration/nodes.py**`**: Individual graph node execution functions (ingestion, aoι, detection, reconciliation, policy, advisory, evidence, audit) with exception trapping.
- **`**src/orchestration/graph.py**`**: Assembles the stateful Directed Acyclic Graph (DAG) connecting all pipeline stages.
- **`**src/agent/models.py**`**: Data models for advisory LLM prompts, requests, and structured responses.
- **`**src/agent/advisory_service.py**`**: Client querying local Ollama runtime (**qwen2.5:7b**), with instant 0ms deterministic rule-based fallback.

13. Evidence Packaging & Persistence (**`src/evidence/`**, **`src/db/`**, **`src/reporting/`**)

- **`**src/evidence/crops.py**`**: Generates 300 DPI high-resolution visual crops of the excavation site plus 15m context margin.
- **`**src/evidence/engine.py**`**: Compiles visual evidence packages including whole-sheet plans, neon overlays, and GeoJSON features.
- **`**src/evidence/completeness.py**`**: Verifies evidence package completeness before release.
- **`**src/reporting/generator.py**`**: Exports PDF and JSON job reports summarizing findings, gate states, and audit trails.
- **`**src/db/engine.py**`**: SQLAlchemy engine configuration connecting to SQLite **safedig.db** with WAL (write-ahead logging) mode.
- **`**src/db/models.py**`**: ORM models for jobs, enquiry records, documents, gate evaluations, and engineer overrides.
- **`**src/db/persistence.py**`**: Database persistence layer writing job states and SHA-256 cryptographic signatures.
- **`**src/db/repositories.py**`**: Data access repositories providing clean query interfaces.

14. Batch Workers & Evaluation Benchmarks (**`src/batch/`** & **`src/eval/`**)

- **`**src/batch/models.py**`**: Data models for batch jobs, task priorities, and execution statuses.
- **`**src/batch/queue.py**`**: Thread-safe in-memory task queue managing concurrent batch runs.
- **`**src/batch/scanner.py**`**: Directory scanner discovering unprocessed job folders in **Data/**.
- **`**src/batch/worker.py**`**: Background worker process consuming jobs from the queue.
- **`**src/eval/models.py**`**: Data models for ground-truth benchmark datasets.
- **`**src/eval/dataset.py**`**: Dataset loader parsing historical verified excavation jobs.
- **`**src/eval/metrics.py**`**: Evaluates precision, recall, false-negative rate, and zero-escaped-hazard compliance.
- **`**src/eval/benchmark.py**`**: Benchmark runner comparing pipeline performance against baseline datasets.

15. QA Workspace & REST API ([src/qa/](#) & [src/api/](#))

- **`**src/qa/workspace.py**`**: Assembles workspace state models, pre-renders map crops, and processes safety engineer overrides.
 - **`**src/qa/disposition.py**`**: Manages human disposition decisions (**`ACCEPT_RISK`**, **`REJECT_PERMIT`**, **`RERUN_SCAN`**).
 - **`**src/api/app.py**`**: FastAPI application entry point configuring CORS, middleware, static files, and routing.
 - **`**src/api/middleware.py**`**: Request logging, timing metrics, and error handling middleware.
 - **`**src/api/schemas.py**`**: REST API request and response schemas.
 - **`**src/api/routes/jobs.py**`**: Endpoints for job upload, execution, and status polling (**`/api/jobs`**).
 - **`**src/api/routes/qa.py**`**: Endpoints serving QA workspace state and accepting sign-offs (**`/api/qa`**).
 - **`**src/api/routes/evidence.py**`**: Streams cropped map images and vector overlays (**`/api/evidence`**).
 - **`**src/api/routes/batch.py**`**: Manages batch job scheduling and queue status (**`/api/batch`**).
 - **`**src/api/routes/catalogue.py**`**: Serves provider warning catalogue data (**`/api/catalogue`**).
 - **`**src/api/routes/eval.py**`**: Triggers evaluation benchmarks and serves accuracy metrics (**`/api/eval`**).
 - **`**src/api/routes/health.py**`**: System health check endpoint (**`/health`**).
 - **`**src/api/routes/metrics.py**`**: Prometheus-compatible operational metrics (**`/metrics`**).
 - **`**src/api/static/index.html**`**: Single-page dashboard featuring radar scanner, gate status grid, and canvas map viewer.
 - **`**src/api/static/app.js**`**: Frontend JavaScript managing canvas zoom/pan, dynamic gate badges, and override actions.
 - **`**src/api/static/styles.css**`**: Tailwind CSS and high-contrast dark theme styling.
-

16. Cross-Platform Utilities ([src/utils/](#))

- **`**src/utils/paths.py**`**: Self-healing cross-machine path resolver. Translates foreign machine paths to local host environment dynamically.
- **`**src/utils/security.py**`**: Input sanitization, path traversal prevention, and filename escaping.
- **`**src/utils/telemetry.py**`**: Execution latency timers, memory profilers, and error counters.
- **`**src/utils/profiler.py**`**: Code block profiling utility identifying processing bottlenecks.
- **`**src/utils/logging.py**`**: Helper logging functions and formatting helpers.